


```
hello world!
[ERROR 0]stext: 0x0000000080200000
[WARN 0]etext: 0x0000000080201000
[INFO 0]sroda: 0x0000000080201000
[DEBUG 0]eroda: 0x0000000080202000
[DEBUG 0]sdata: 0x0000000080202000
[INFO 0]edata: 0x0000000080202000
[WARN 0]sbss : 0x0000000080212000
[ERROR 0]ebss : 0x0000000080212000
[PANIC 0] os/main.c:39: ALL DONE
```

退出 QEMU: 按 `Ctrl+A`, 再按 `X`

二、代码框架分析

第一次打开这个项目的时候, 看到一堆 `.S`、`.ld`、`.c` 文件, 完全不知道从哪里开始看。后来发现了一个窍门: 从 `make run` 的执行流程倒推, 就能理清代码的组织结构。

2.1 项目结构

```
uCore-Tutorial-Code-2025s/
├─ bootloader/
│   └─ rustsbi-qemu.bin    # RustSBI 引导程序
├─ os/
│   ├─ entry.S            # 内核入口 (汇编)
│   ├─ main.c             # 主函数
│   ├─ kernel.ld          # 链接脚本
│   ├─ sbi.c/h            # SBI 调用封装
│   ├─ console.c/h        # 控制台输出
│   ├─ printf.c/h         # 格式化输出
│   ├─ log.h              # 彩色日志宏
│   ├─ riscv.h            # RISC-V 寄存器操作
│   ├─ types.h            # 类型定义
│   └─ defs.h             # 常量定义
├─ Makefile               # 构建脚本
└─ README.md
```

2.2 启动流程

一开始看到指导书里的"启动流程", 什么 M态、S态, 感觉像在看天书。后来查了资料才知道, 这些"态"就是 CPU 的不同权限等级, 就像公司里的员工、经理、老板, 能做的事情不一样。



1. **QEMU 启动**: CPU 从 0x1000 开始执行固化代码

2. **跳转 RustSBI**: 跳转到 0x80000000, 完成 M 态初始化
3. **进入内核**: 跳转到 0x80200000, 执行 `_entry` 函数

2.3 关键文件分析

下面这几个文件是本章的核心, 刚开始看的时候一头雾水, 尤其是 `.ld` 和 `.s` 文件, 完全没接触过。花了不少时间查资料才搞明白它们各自的作用。

kernel.ld - 链接脚本

这个文件一开始完全看不懂, 什么 `SECTIONS`、`BASE_ADDRESS`... 后来才知道, 链接脚本就是告诉编译器"把代码放在内存的哪个位置"。0x80200000 这个地址不是随便写的, 是 RustSBI 约定好的内核入口地址。

```
BASE_ADDRESS = 0x80200000;
SECTIONS
{
    . = BASE_ADDRESS;
    skernel = .;

    stext = .;
    .text : {
        *(.text.entry) /* 第一行代码 */
        *(.text .text.*)
    }
    /* ... 其他段 ... */
}
```

entry.S - 内核入口

汇编代码看起来很吓人, 但其实这个文件做的事情很简单: 设置一下栈, 然后跳转到 C 语言的 `main` 函数。为什么需要汇编? 因为刚启动的时候连栈都没有, C 语言函数没法运行 (函数调用需要栈来保存返回地址和局部变量)。

```
.section .text.entry
.global _entry
_entry:
    la sp, boot_stack_top    # 设置栈指针
    call main                # 调用 main 函数

.section .bss.stack
boot_stack:
    .space 4096 * 16         # 64KB 启动栈
boot_stack_top:
```

main.c - 主函数

终于看到熟悉的 C 代码了! 这个文件就好理解多了。不过有个小细节让我困惑了一会儿: 为什么要 `clean_bss()`? 查了资料才知道, BSS 段存放的是未初始化的全局变量, C 语言规定它们默认值是 0, 但硬件启动时内存里是随机值, 所以需要手动清零。

```

void main()
{
    clean_bss();           // 清空 bss 段
    console_init();        // 初始化控制台
    printf("\n");
    printf("hello world!\n");
    errorf("stext: %p", s_text);
    // ... 输出内存布局 ...
    panic("ALL DONE");    // 关机
}

```

sbi.c - SBI 调用

这个文件让我第一次接触到"系统调用"的概念。简单来说，内核想输出一个字符，自己做不到（没有直接操作硬件的代码），就通过 `ecall` 指令请求 RustSBI 帮忙。RustSBI 运行在更高权限的 M 态，可以直接操作硬件。

```

void console_putchar(int c)
{
    sbi_call(SBI_CONSOLE_PUTCHAR, c, 0, 0);
}

void shutdown()
{
    sbi_call(SBI_SHUTDOWN, 0, 0, 0);
}

```

2.4 Makefile 分析

关键编译参数：

```

QEMU = qemu-system-riscv64
QEMUOPTS = \
    -nographic \           # 无图形界面
    -smp 1 \                # 单核
    -machine virt \         # RISC-V VirtIO Board
    -bios $(BOOTLOADER) \   # 使用 RustSBI
    -kernel kernel          # 加载内核

```

三、核心概念理解

本章涉及不少新概念，刚开始学的时候很容易混淆。这里记录一下我的理解，方便以后回顾。

3.1 系统调用 (syscall)

这个词在后面的章节会反复出现。本章的理解是：当一段代码想做自己权限不够的事情时，就通过 `ecall` 指令"请求上级帮忙"。本章是内核请求 RustSBI，后面还会看到用户程序请求内核。

3.2 特权级

本章只用到了 M 态和 S 态，U 态（用户态）要到 Lab2 才会接触。现在先记住：权限从高到低是 $M > S > U$ ，高权限的代码可以做更多事情。

特权级	名称	运行内容
M 态	Machine Mode	RustSBI
S 态	Supervisor Mode	操作系统内核
U 态	User Mode	用户程序

3.3 内存布局

运行结果里那些 `s.text`、`e.text` 之类的输出，一开始完全不知道是什么意思。后来明白了：这些是链接脚本定义的符号，标记了各个段的起始和结束地址。`s` 开头表示 start，`e` 开头表示 end。

从运行结果可以看到内核的内存布局：

段	起始地址	结束地址	说明
.text	0x80200000	0x80201000	代码段
.rodata	0x80201000	0x80202000	只读数据
.data	0x80202000	0x80202000	已初始化数据
.bss	0x80212000	0x80212000	未初始化数据

四、验证截图

```
hjl@hjl-VMware-Virtual-Platform: ~/桌面/herdream/2025-ucore-riscv-清华...
[rustsbi] Platform Name      : riscv-virtio,qemu
[rustsbi] Platform SMP      : 1
[rustsbi] Platform Memory   : 0x80000000..0x88000000
[rustsbi] Boot HART         : 0
[rustsbi] Device Tree Region : 0x87000000..0x87000ef2
[rustsbi] Firmware Address  : 0x80000000
[rustsbi] Supervisor Address : 0x80200000
[rustsbi] pmp01: 0x00000000..0x80000000 (-wr)
[rustsbi] pmp02: 0x80000000..0x80200000 (---)
[rustsbi] pmp03: 0x80200000..0x88000000 (xwr)
[rustsbi] pmp04: 0x88000000..0x00000000 (-wr)

hello wrold!
[ERROR 0]stext: 0x0000000008020000
[WARN 0]etext: 0x00000000080201000
[INFO 0]sroda: 0x00000000080201000
[DEBUG 0]eroda: 0x00000000080202000
[DEBUG 0]sdata: 0x00000000080202000
[INFO 0]edata: 0x00000000080202000
[WARN 0]sbss : 0x00000000080212000
[ERROR 0]ebss : 0x00000000080212000
[PANIC 0] os/main.c:39: ALL DONE
hjl@hjl-VMware-Virtual-Platform:~/桌面/herdream/2025-ucore-riscv-清华/uCore-Tutorial-Code-2025S-ch1$ ^C
```

五、实验总结

本章完成了以下工作：

- ✓ 成功运行 ch1 分支
- ✓ 理解 QEMU → RustSBI → 内核的启动流程
- ✓ 分析 kernel.ld 链接脚本和内存布局
- ✓ 理解 entry.S 汇编入口
- ✓ 理解 sbi.c 的系统调用封装

收获：本章虽然没有编程任务，但对后续实验至关重要。理解了代码框架后，后续章节只需在此基础上逐步添加功能。